

AEP-NEM Transparent Engineering Network v0.1

透明工程网络与无皇帝原则 - AI 原生工程体系的去中心化宪章草案

Raphael / WenChuan Research

2026-06-02

目录

1	AEP-NEM Transparent Engineering Network v0.1	3
1.1	0. 概念锁定	3
2	1. 为什么 AEP-NEM 必须走向去中心化	4
2.1	1.1 中心化任务平台的局限	4
2.2	1.2 协议比平台更重要	5
2.3	1.3 AI 时代任务外包的根本问题	5
3	2. 术语体系	5
4	3. TEN 的正式定义	6
4.1	3.1 基础定义	6
4.2	3.2 TEN 不是反秩序，而是反单点垄断	7
4.3	3.3 TEN 的一句话定义	7
5	4. No Emperor Principle / 无皇帝原则	7
5.1	4.1 原则定义	7
5.2	4.2 无皇帝原则不等于无责任	8
5.3	4.3 无皇帝原则的工程含义	8
6	5. TEN 的六大宪法级原则	9
6.1	5.1 协议优先原则	9
6.2	5.2 多中心发布原则	9
6.3	5.3 可验证做功原则	9
6.4	5.4 分叉自由原则	10
6.5	5.5 开放合并原则	10
6.6	5.6 透明记录原则	10

7	6. TEN 的基本运行模型	11
7.1	6.1 多中心 AEP 发布	11
7.2	6.2 多执行者竞争与协作	11
7.3	6.3 Gate 多方验证	11
7.4	6.4 NEM 采纳与升级	12
8	7. TEN 的工程结构	12
8.1	7.1 从原子到透明网络	12
8.2	7.2 E-Mesh 与 TEN 的区别	13
8.3	7.3 TEN 的核心对象	13
9	8. TEN 与 Git、Docker、区块链、开源社区的关系	13
9.1	8.1 与 Git 的关系	14
9.2	8.2 与 Docker 的关系	14
9.3	8.3 与区块链的关系	14
9.4	8.4 与开源社区的关系	15
10	9. TEN 的状态机	15
10.1	9.1 AEP 状态	15
10.2	9.2 NEM 状态	15
10.3	9.3 E-Chain 状态	16
10.4	9.4 PoEW 状态	16
11	10. TEN 的最小实现路线	16
11.1	10.1 阶段一：本地协议	16
11.2	10.2 阶段二：GitHub 多仓库协作	17
11.3	10.3 阶段三：多中心 Registry	17
11.4	10.4 阶段四：PoEW Ledger	17
11.5	10.5 阶段五：E-Mesh	18
11.6	10.6 阶段六：TEN	18
12	11. TEN 的安全与责任边界	18
12.1	11.1 去中心化不等于无边界	18
12.2	11.2 Gate 必须包含安全检查	18
12.3	11.3 多中心验证不是无标准	19
13	12. TEN 的治理模型	19
13.1	12.1 协议治理	19
13.2	12.2 多维护者模型	19
13.3	12.3 分叉即治理	20
13.4	12.4 合并即进化	20
14	13. TEN 的标准目录建议	20

1

AEP-NEM TRANSPARENT ENGINEERING NETWORK V0.1

3

15

14. TEN 与人类协作的未来

21

16

15. 核心公理

22

16.1

公理一：任务必须可封装

22

16.2

公理二：做功必须可证明

22

16.3

公理三：验证必须可复核

22

16.4

公理四：节点必须可进化

22

16.5

公理五：分叉必须自由

22

16.6

公理六：合并必须开放

22

16.7

公理七：协议必须高于平台

22

16.8

公理八：透明必须高于权威

23

17

16. 最终概念锁定

23

18

17. 下一步研究方向

23

19

18. 简短宣言版

24

1 AEP-NEM Transparent Engineering Network v0.1

中文名：AEP-NEM 透明工程网络

英文名：AEP-NEM Transparent Engineering Network

简称：TEN

核心原则：No Emperor Principle / 无皇帝原则

状态：Concept Lock / 宪章草案 v0.1

1.1 0. 概念锁定

AEP-NEM Engineering System 的极限形态，不是一个中心化任务平台，不是某个公司、国家、政府、实验室或中心机构发布任务、解释规则、验证结果、分配权力的系统。

它的极限形态应当是：

TEN = Transparent Engineering Network

透明工程网络

TEN 是一个面向 AI 协作时代的去中心化工程协作网络。它以 AEP 为工程原子，以 NEM 为节点进化分子，以 E-Chain 连接主线与支线，以 E-Organism 承载复杂工程生命体，以 PoEW 与 Gate 建立可信验证，以 E-Mesh 形成多中心工程网格，最终走向透明、开放、可验证、可分叉、可合并、可持续进化的全人类工程协作系统。

它的最高原则是：

No Emperor Principle

无皇帝原则

这句话可以被写成更具精神性的表达：

这个世界不应该有皇帝。

在 AEP-NEM 的语境中，它不是情绪口号，而是协议设计原则：**任何单一中心都不应永久垄断任务发布权、协议解释权、验证权、采纳权、分叉权和工程秩序的最终裁判权。**

2 1. 为什么 AEP-NEM 必须走向去中心化

2.1 1.1 中心化任务平台的局限

如果 AEP-NEM 最后只是变成一个中心化平台，它会退化成旧时代的任务外包平台：

中心发布任务

↓

人或 AI 接单执行

↓

中心审核结果

↓

中心决定采纳与否

这种模式的问题在于：

1. 任务发布权被中心垄断。
2. 协议解释权被中心垄断。
3. 验证结果依赖中心仲裁。
4. 执行者变成平台雇佣关系的附属。
5. 创造性分叉会被平台规则压制。
6. 工程秩序可能被商业、政治、流量、审查或组织利益扭曲。

这不是 AI 时代真正需要的工程协作结构。

AI 时代真正需要的是一种**协议优先、平台其次**的工程协作方式。

2.2 1.2 协议比平台更重要

平台是某个组织运行的产品。协议是一套可被多个组织、多个工具、多个网络共同实现的规则。

AEP-NEM 的未来不应是：

一个平台统治所有工程任务。

而应是：

一套开放协议，被许多平台、工具、AI、团队、个人和组织共同实现。

因此，AEP-NEM 的设计应当遵循：

Protocol First, Platform Second.

协议优先，平台其次。

平台可以有很多个，但协议不应被任何一个平台私有化。

2.3 1.3 AI 时代任务外包的根本问题

AI 时代最重要的问题不是“有没有人执行任务”，而是：

任务是否被清楚封装？

执行过程是否可追踪？

结果是否可验证？

贡献是否可证明？

模块是否可复用？

系统是否可持续进化？

AEP-NEM 的去中心化价值就在于，它把工程协作从“信任某个中心”转向“信任可验证的工程结构”：

任务结构可信

执行日志可信

输出结果可信

验收规则可信

工作量证明可信

分叉合并记录可信

3 2. 术语体系

层级	缩写	英文命名	中文命名	核心含义
最小做功单元	AEP	Atomic Engineering Package	工程原子包	最小可封装、可执行、可验证的工程做功单元
节点模块	NEM	Node Evolution Molecule	节点进化分子	多个 AEP 组合形成的可进化工程模块
主线/支线	E-Chain	Engineering Chain	工程链	多个 NEM 按依赖关系连接形成的工程路径
项目整体	E-Organism	Engineering Organism	工程生命体	多条 E-Chain 构成的复杂工程系统
协作网络	E-Mesh	Engineering Mesh	工程网络	多个工程生命体、多中心参与者组成的协作网络
极限形态	TEN	Transparent Engineering Network	透明工程网络	不服从单一中心的去中心化工程协作网络
工作量证明	PoEW	Proof of Engineering Work	工程工作量证明	执行过程与结果的可追踪证明
验收闸门	Gate	Evolution Gate	进化闸门	判断 AEP/NEM/E-Chain 是否可采纳、升级或重构

核心层级关系：

AEP -> NEM -> E-Chain -> E-Organism -> E-Mesh -> TEN
工程原子 -> 节点进化分子 -> 工程链 -> 工程生命体 -> 工程网络 -> 透明工程网络

4 3. TEN 的正式定义

4.1 3.1 基础定义

TEN (Transparent Engineering Network, 透明工程网络) 是 AEP-NEM Engineering System 的极限形态。

它是一种去中心化、多中心、开放验证、开放分叉、开放合并的工程协作网络。任何人、AI、团队、社区、实验室、公司或组织都可以发布 AEP、维护 NEM、连接 E-Chain、构建 E-Organism，并通过 PoEW 与 Gate

完成可信验证。

TEN 不服从任何单一国家、政府、公司、平台、基金会或中心机构的永久垄断。它服从的是开放协议、透明规则、可验证做功和多中心复核。

4.2 3.2 TEN 不是反秩序，而是反单点垄断

TEN 的去中心化不是无规则，也不是混乱。它反对的不是秩序，而是**单点垄断的秩序**。

TEN 需要更高密度的工程秩序：

开放标准
明确任务
透明输入
可追踪输出
可复核验证
可记录失败
可分叉演化
可合并采纳

也就是说，TEN 并不是“没有规则”，而是让规则从中心授权转向协议公开，让协作从服从平台转向遵守可验证协议。

4.3 3.3 TEN 的一句话定义

TEN 是一种不服从单一中心、由 AEP 工程原子和 NEM 节点进化分子构成的透明工程协作网络。

更完整的定义：

TEN 是 AI 时代的去中心化工程协作网络。它允许任何人、AI、团队和组织以 AEP 的形式发布工程原子

5 4. No Emperor Principle / 无皇帝原则

5.1 4.1 原则定义

无皇帝原则是 TEN 的最高设计原则。

它可以被定义为：

AEP-NEM 不承认任何单一中心对协议、任务、验证、采纳、分叉和合并拥有永久垄断权。

这意味着：

1. 没有唯一任务发布者。
2. 没有唯一协议解释者。
3. 没有唯一验证者。
4. 没有唯一采纳者。
5. 没有唯一工程中心。
6. 没有不可挑战的最终权威。

5.2 4.2 无皇帝原则不等于无责任

无皇帝原则并不意味着逃避责任、逃避安全、逃避法律或鼓励破坏性行为。

它真正反对的是：

单一中心垄断工程协作的解释权和命运。

在 AEP-NEM 的设计中，任何发布、执行、验证和采纳都应尽可能留下记录：

谁发布了 AEP
谁执行了 AEP
谁生成了 PoEW
谁执行了 Gate
谁采纳了结果
谁分叉了 NEM
谁合并了 E-Chain

去中心化不是无责任，而是多中心责任。

5.3 4.3 无皇帝原则的工程含义

无皇帝原则在工程上意味着：

维度	中心化模式	无皇帝模式
任务发布	中心发布任务	多中心发布 AEP
协议解释	平台最终解释	开放规范与版本化讨论
结果验证	中心审核	PoEW + Gate + 多方复核
模块演化	中心路线图	NEM 分叉、合并、重构
项目归属	平台所有	E-Organism 可多中心维护
协作网络	用户服从平台	参与者服从协议

6 5. TEN 的六大宪法级原则

6.1 5.1 协议优先原则

Protocol First, Platform Second.

任何平台、工具、网站、App、CLI 或 SaaS 都只是 AEP-NEM 协议的一种实现。平台不能高于协议，工具不能垄断格式，服务商不能拥有唯一解释权。

6.2 5.2 多中心发布原则

任何主体都可以发布 AEP：

个人
AI Agent
研究团队
开源社区
公司
实验室
教育机构
跨国协作组
匿名贡献者

AEP 的价值不由发布者身份决定，而由其是否清晰、可执行、可验证、可复用决定。

6.3 5.3 可验证做功原则

工程贡献的可信度不应来自身份崇拜，而应来自：

PoEW = Proof of Engineering Work
工程工作量证明

一个 AEP 是否完成，不应只看执行者说了什么，而应看：

有没有输入记录
有没有执行记录
有没有输出结果
有没有日志
有没有失败分类
有没有 validation report
有没有 Gate 判断
有没有可复现路径

6.4 5.4 分叉自由原则

任何 NEM、E-Chain、E-Organism 都应允许被分叉。

分叉不是背叛，而是工程演化机制。

当一个中心路线出现问题，新的团队可以：

fork NEM

修改 AEP 序列

重新定义 Gate

生成新的 PoEW

形成新的 E-Chain

分叉自由是防止工程皇帝出现的核心机制。

6.5 5.5 开放合并原则

分叉之后，还应允许合并。

好的工程结果不应该被身份、组织、派系阻断，而应该通过明确标准重新进入公共工程链：

提交 PoEW

通过 Gate

进入候选分支

多方复核

合并进入 NEM / E-Chain

TEN 不是鼓励永久分裂，而是允许分叉、竞争、验证和重新合并。

6.6 5.6 透明记录原则

TEN 的透明不是要求所有隐私公开，而是要求工程过程中的关键判断可追踪：

任务从哪里来

谁执行过

产生了什么结果

失败在哪里

谁验证过

为什么采纳

为什么拒绝

为什么 HOLD

为什么重构

没有透明记录，就没有真正的工程信任。

7 6. TEN 的基本运行模型

7.1 6.1 多中心 AEP 发布

在 TEN 中，AEP 可以由多个中心发布：

Research Lab A 发布 AEP-001

Open Community B 发布 AEP-002

AI Agent C 生成 AEP-003 草案

Company D 发布 AEP-004

Independent Developer E 发布 AEP-005

所有 AEP 都可以进入公开索引，但进入某个 NEM 之前必须通过对应的 Gate。

7.2 6.2 多执行者竞争与协作

同一个 AEP 可以被多个执行者处理：

AEP-X

-> Executor 1 执行，产生 PoEW-1

-> Executor 2 执行，产生 PoEW-2

-> AI Agent 3 执行，产生 PoEW-3

-> Team 4 执行，产生 PoEW-4

多个 PoEW 可以被比较、验证、复核和采纳。

这会让 AEP 不再是一次性委托，而变成可竞争、可复现、可改进的工程原子。

7.3 6.3 Gate 多方验证

Gate 不应由单一中心永久控制。

一个 Gate 可以有多种验证方式：

自动测试 Gate

人工审查 Gate

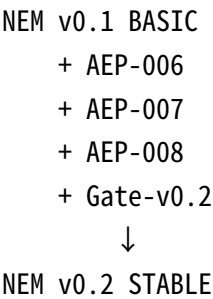
专家评审 Gate

社区复核 Gate
AI 交叉验证 Gate
长期运行 Gate
安全审查 Gate

不同 NEM 可以定义不同 Gate，但 Gate 的规则应当尽可能公开。

7.4 6.4 NEM 采纳与升级

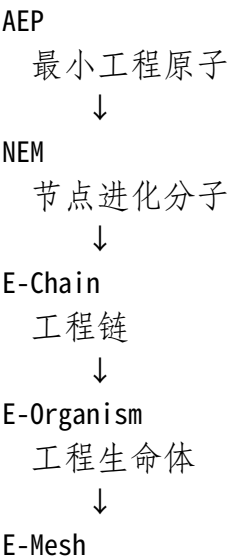
当一组 AEP 通过 Gate 后，可以推动某个 NEM 升级：

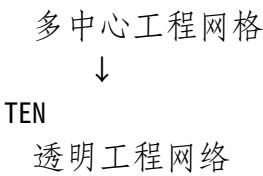


NEM 的成熟不是一次性完成，而是不断吸收 AEP 工程原子、生成 PoEW、通过 Gate、完成版本进化。

8 7. TEN 的工程结构

8.1 7.1 从原子到透明网络





8.2 7.2 E-Mesh 与 TEN 的区别

E-Mesh 是工程网络，强调多个工程生命体之间的连接。

TEN 是透明工程网络，强调协议层面的去中心化原则、开放验证机制和无皇帝原则。

关系可以理解为：

E-Mesh 是网络形态。

TEN 是网络宪章与极限形态。

8.3 7.3 TEN 的核心对象

TEN 至少包含以下对象：

对象	含义
AEP Registry	工程原子索引
NEM Registry	节点进化分子索引
PoEW Ledger	工程工作量证明记录
Gate Registry	验收闸门规则库
E-Chain Map	工程链地图
E-Organism Archive	工程生命体档案
Fork Graph	分叉关系图
Merge Record	合并记录
Trust Profile	执行者/验证者可信记录

这些对象不一定由一个中心托管。它们可以被多个实现、多个镜像、多个社区共同维护。

9 8. TEN 与 Git、Docker、区块链、开源社区的关系

TEN 不是这些系统的简单复制，但可以吸收它们的结构智慧。

9.1 8.1 与 Git 的关系

Git 提供了分布式版本管理思想。

TEN 可以吸收：

- fork
- branch
- merge
- commit history
- diff
- review

但 TEN 不只管理代码，还管理任务、执行、验证、模块进化和工程生命体。

9.2 8.2 与 Docker 的关系

Docker 封装运行环境和应用。

TEN 可以吸收：

- containerization
- versioned image
- portable module
- dependency declaration

但 TEN 的核心不是运行容器，而是工程做功容器与协作协议。

9.3 8.3 与区块链的关系

区块链强调分布式记录和工作量证明。

TEN 可以吸收：

- proof
- ledger
- verification
- anti-tampering
- multi-party trust

但 TEN 不应被简化为发币或金融系统。TEN 的核心是工程协作，而不是金融投机。

9.4 8.4 与开源社区的关系

开源社区提供了开放协作、分叉、合并和社区治理的经验。

TEN 可以吸收：

- issue
- pull request
- maintainer
- review
- license
- community governance

但 TEN 进一步强调 AI 可读的工程包、PoEW、Gate 和 NEM 进化结构。

10 9. TEN 的状态机

10.1 9.1 AEP 状态

DRAFT	草案
READY	可执行
RUNNING	执行中
PASS	单项通过
FAIL	执行失败
HOLD	暂停，需要补充条件
ADOPTED	被某个 NEM 采纳
FORKED	被分叉
ARCHIVED	归档

10.2 9.2 NEM 状态

DRAFT	草案模块
BASIC	基础可用
STABLE	稳定可用
EXTENDED	增强可用
ADOPTED	主线采纳
FORKED	已被分叉
MERGED	分叉成果已合并

REFACTORED	重构完成
DEPRECATED	旧版弃用
ARCHIVED	归档

10.3 9.3 E-Chain 状态

PLANNED	规划中
ACTIVE	运行中
BRANCHED	已分支
MERGING	合并中
STABLE	稳定主线
SUSPENDED	暂停
ARCHIVED	归档

10.4 9.4 PoEW 状态

SUBMITTED	已提交
VALIDATING	验证中
VERIFIED	已验证
DISPUTED	存在争议
REJECTED	被拒绝
SUPERSEDED	被更优证明取代

11 10. TEN 的最小实现路线

TEN 不需要一开始就开发庞大的去中心化网络。

最小阻力路线是：

本地协议 -> GitHub 多仓库协作 -> 多中心 Registry -> PoEW 索引 -> E-Mesh -> TEN

11.1 10.1 阶段一：本地协议

使用现有技术：

ZIP
Markdown

YAML

JSON

PDF

Git

Python CLI

先定义 AEP、NEM、PoEW、Gate 的格式。

11.2 10.2 阶段二：GitHub 多仓库协作

使用 GitHub 作为早期协作场：

一个仓库维护 AEP 标准

一个仓库维护 NEM 标准

一个仓库维护 TEN 宪章

多个仓库发布具体 AEP/NEM 示例

GitHub 不是最终中心，只是早期工具。

11.3 10.3 阶段三：多中心 Registry

不同社区可以维护自己的 registry：

AEP Registry CN

AEP Registry Global

NEM Registry Research

NEM Registry Music AI

NEM Registry Software Engineering

Registry 之间可以互相引用、镜像、分叉和合并。

11.4 10.4 阶段四：PoEW Ledger

建立工程工作量证明索引：

PoEW ID

AEP ID

Executor ID

Input Hash

Output Hash

Report Hash

Gate Result
Timestamp
Reviewer Signature

这里不必一开始上链。可以先用 JSONL、Git、SQLite、对象存储或内容哈希。

11.5 10.5 阶段五：E-Mesh

当多个 E-Organism 之间开始共享 AEP、NEM、PoEW 和 Gate，就形成 E-Mesh。

11.6 10.6 阶段六：TEN

当 E-Mesh 进一步具备去中心化发布、验证、分叉、合并、记录和自治能力，就进入 TEN。

12 11. TEN 的安全与责任边界

12.1 11.1 去中心化不等于无边界

TEN 应当坚持：

- 不鼓励破坏性工程
- 不鼓励恶意代码
- 不鼓励欺骗性 PoEW
- 不鼓励伪造验证
- 不鼓励侵犯他人数据和隐私
- 不鼓励违反安全与伦理边界的任务

去中心化不是逃避责任，而是防止单点垄断。

12.2 11.2 Gate 必须包含安全检查

重要 NEM 的 Gate 应包含：

- 安全检查
- 权限检查
- 数据保护检查
- 依赖风险检查

不可逆操作检查
敏感输出检查

12.3 11.3 多中心验证不是无标准

多中心验证需要明确标准。

不同社区可以拥有不同 Gate，但 Gate 必须清楚声明：

验证什么
不验证什么
通过条件
失败条件
HOLD 条件
复核机制
争议处理方式

13 12. TEN 的治理模型

13.1 12.1 协议治理

AEP-NEM 协议本身应采用开放版本治理：

AEP Standard v0.1
NEM Standard v0.1
AEP-NEM System v0.1
TEN Charter v0.1

每个版本都应保留变更记录。

13.2 12.2 多维护者模型

协议不应依赖单一维护者。

可以形成：

Core Maintainers
Domain Maintainers
Gate Reviewers

PoEW Auditors
Registry Operators
Community Fork Maintainers

13.3 12.3 分叉即治理

在 TEN 中，分叉本身就是治理机制。

当协议路线出现分歧时，不应强迫所有人服从唯一中心，而应允许：

协议分叉
Gate 分叉
NEM 分叉
E-Chain 分叉
Registry 分叉

更好的分支会通过可验证成果获得采纳。

13.4 12.4 合并即进化

治理的目标不是永远分裂，而是在充分验证后重新合并优秀成果。

Fork -> Experiment -> PoEW -> Gate -> Review -> Merge

14 13. TEN 的标准目录建议

一个 TEN Charter 仓库可以采用以下结构：

```
ten-charter/
├── README.md
├── docs/
│   ├── TEN_Charter_v0.1.md
│   ├── TEN_Charter_v0.1.pdf
│   ├── AEP_NEM_Engineering_System_v0.1.md
│   └── glossary.md
├── standards/
│   ├── aep_standard.md
│   ├── nem_standard.md
│   └── poew_standard.md
```

```

|   |—— gate_standard.md
|   |—— registry_standard.md
|—— schemas/
|   |—— aep_manifest.schema.json
|   |—— nem_manifest.schema.json
|   |—— poew.schema.json
|   |—— gate.schema.json
|—— examples/
|   |—— sample_aep/
|   |—— sample_nem/
|   |—— sample_poew/
|—— research/
|   |—— decentralization.md
|   |—— governance.md
|   |—— validation.md
|   |—— fork_merge_model.md
|—— CHANGELOG.md

```

15 14. TEN 与人类协作的未来

TEN 的真正目标不是让 AI 取代人类协作，而是让人类与 AI 的协作更清楚、更透明、更可验证。

传统协作依赖：

会议
口头沟通
邮件
模糊外包
中心审核
人情信任

TEN 试图转向：

AEP 封装任务
NEM 组织模块
PoEW 证明做功
Gate 验证结果
E-Chain 连接方向
E-Organism 承载系统

E-Mesh 扩展协作网络

TEN 维持无皇帝原则

这是一种新的工程文明秩序。

16 15. 核心公理

16.1 公理一：任务必须可封装

如果一个任务不能被封装，它就难以被 AI 稳定执行。

16.2 公理二：做功必须可证明

如果做功不能被证明，它就不能成为可信工程贡献。

16.3 公理三：验证必须可复核

如果验证不能被复核，它就会退化为中心权威。

16.4 公理四：节点必须可进化

如果节点不能进化，它就只是一次性任务，不是工程模块。

16.5 公理五：分叉必须自由

如果不能分叉，系统最终会产生皇帝。

16.6 公理六：合并必须开放

如果不能合并，系统最终会碎片化。

16.7 公理七：协议必须高于平台

如果平台高于协议，开放协作最终会被平台私有化。

16.8 公理八：透明必须高于权威

如果权威高于透明，工程协作最终会变成服从关系。

17 16. 最终概念锁定

AEP-NEM Engineering System 的极限不是一个中心化任务平台，而是 TEN: Transparent Engineering Network。

TEN 是一个不服从单一国家、政府、公司、平台或中心机构的去中心化工程协作网络。它以 AEP 为工程原子，以 NEM 为节点进化分子，以 E-Chain 组织工程路径，以 E-Organism 承载复杂项目，以 E-Mesh 扩展多中心协作，并通过 PoEW 与 Gate 建立可信验证。

它的最高原则是：

No Emperor Principle

无皇帝原则

精神表达是：

这个世界不应该有皇帝。

工程表达是：

任何单一中心都不应永久垄断任务发布、协议解释、结果验证、模块采纳、分叉合并和工程秩序的最

AEP-NEM 的未来，应当是一个透明、开放、多中心、可验证、可分叉、可合并、可持续进化的工程网络。

这不是普通项目管理系统。

这是 AI 时代的工程协作宪章。

18 17. 下一步研究方向

1. 起草 TEN Charter v0.2，进一步细化无皇帝原则。
2. 设计 PoEW Ledger 的最小 JSONL 格式。
3. 设计 Gate Federation，支持多方验证者。
4. 设计 AEP Registry 与 NEM Registry 的多中心结构。
5. 设计 Fork / Merge Graph，记录 NEM 与 E-Chain 演化。
6. 设计 Trust Profile，记录执行者与验证者的历史贡献。

7. 用 GitHub 多仓库模拟早期 E-Mesh。
 8. 将 Moodify Runtime NEM 作为第一个真实 TEN 试验场。
-

19 18. 简短宣言版

AEP 是工程原子。

NEM 是节点进化分子。

E-Chain 是工程链。

E-Organism 是工程生命体。

E-Mesh 是工程网格。

TEN 是透明工程网络。

AEP-NEM 不服从单一中心。

AEP-NEM 服从开放协议、可验证做功和透明协作。

这个世界不应该有皇帝。